Database Users, Roles, Privileges, and Access Control

Group 8 (Campus Canteen Pre Order and Queue Management System)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** Campus Canteen Pre Order and Queue Management System **Repository Link:** **https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System.git** **Our Group Repository:**
Projects set up	**The Campus Canteen Pre-Order and Queue Management System repository was used as the basis for implementing the database access-control plan. The project is a campus canteen management application designed to support food pre-ordering, customer transactions, and queue management operations.** **The first step was to examine the existing project files and understand how the application connects to its database. The repository structure and available database-related files were reviewed to identify the database configuration, existing tables, and other database objects that would require access restrictions.** **The database used for this activity was named grp08_bwisITdb, with the default schema public. The existing database structure was retained, and the access-control implementation was prepared separately to avoid modifying the original system functionality.** **The database was inspected using PostgreSQL queries to identify existing schemas, tables, columns, sequences, and database roles. This information was used as the basis for preparing the role assignments and privilege configuration.** **After reviewing the database setup and access-control requirements, a separate folder named activity4 was prepared to organize the SQL scripts and documentation.** **The following files were created:** • **00_inspect_database.sql** • **01_create_roles.sql**

	• **02_grant_privileges.sql** • **03_revoke_privileges.sql** • **04_test_privileges.sql** • **access_control_plan.md** • **screenshots/** **These files were organized to make the implementation easier to understand, execute, verify, and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each user role. The implementation follows the Principle of Least Privilege, meaning that each role should only receive the permissions necessary for its assigned responsibilities.** **Four PostgreSQL group roles were established to represent the different responsibilities within the Campus Canteen system.** **Administrator Role (canteen_admin_role)** **The administrator role is responsible for managing and maintaining the Campus Canteen database. It receives ALL PRIVILEGES on the existing tables and sequences within the public schema.** **Its responsibilities include viewing database records, inserting new records, updating existing information, deleting records when necessary, and managing database access permissions.** **This role is intended only for authorized database administrators and should not be used for ordinary application operations.** **Application Role (canteen_app_role)** **The application role is intended for the Campus Canteen application that interacts with the database. It allows the application to perform normal database operations required by the system.** **Its responsibilities include retrieving information needed by the application, adding new records, and updating existing information.** **The application role receives SELECT, INSERT, and UPDATE privileges on all existing tables in the public schema. It also receives USAGE and SELECT privileges on existing sequences to support database operations.**

	It is restricted from performing DELETE and TRUNCATE operations to help prevent unnecessary removal of database records. It is also restricted from creating database objects in the public schema. **Cashier Role (canteen_cashier_role)** **The cashier role is intended for personnel responsible for handling customer orders and payment-related transactions within the Campus Canteen system.** **Its responsibilities include viewing relevant order information, adding transaction records, and updating existing transaction information.** **The cashier role receives SELECT, INSERT, and UPDATE privileges on all existing tables in the public schema. It also receives USAGE and SELECT privileges on existing sequences.** **It is restricted from performing DELETE, TRUNCATE, REFERENCES, and TRIGGER operations. It is also restricted from creating database objects in the public schema.** **The cashier role should eventually be limited to the specific order and payment tables required for its responsibilities rather than receiving access to every table.** **Read-Only Role (canteen_readonly_role)** **The read-only role is intended for authorized users who only need to view database information.** **Its responsibilities include viewing records, inspecting available database information, and reviewing information for monitoring and reporting purposes.** **The read-only role receives SELECT privileges on all existing tables in the public schema.** **It cannot insert, update, delete, or truncate records. It is also restricted from creating objects in the public schema.** **This ensures that users assigned to this role can inspect database information without making unauthorized modifications.** --- ## Access-Control Matrix

	The following matrix summarizes the intended database permissions assigned to each group role.

Database Operations	Administrator	Application	Cashier	Read-Only
CONNECT	**Allowed**	**Allowed**	**Allowed**	**Allowed**
SELECT	**Allowed**	**Allowed**	**Allowed**	**Allowed**
INSERT	**Allowed**	**Allowed**	**Allowed**	**Denied**
UPDATE	**Allowed**	**Allowed**	**Allowed**	**Denied**
DELETE	**Allowed**	**Denied**	**Denied**	**Denied**
TRUNCATE	**Allowed**	**Denied**	**Denied**	**Denied**
CREATE in public schema	**Not explicitly granted**	**Denied**	**Denied**	**Denied**
Sequence USAGE	**Allowed**	**Allowed**	**Allowed**	**Not granted**

The permissions listed above represent the intended access-control

configuration based on the prepared SQL scripts. Actual permissions must be verified against the implemented database privileges.

Database Users and Role Assignments

The following database login accounts were prepared:

Database User	Assigned Role
canteen_admin	canteen_admin_role
canteen_app	canteen_app_role
canteen_cashier	canteen_cashier_role
canteen_viewer	canteen_readonly_role

The four group roles were created with the NOLOGIN attribute. This means they function as permission groups and cannot be used as direct login accounts.

The login accounts were created using the LOGIN attribute and assigned to their corresponding group roles through PostgreSQL GRANT statements.

This allows database users to inherit the permissions assigned to their respective roles.

The administrator account is intended for database administration, the application account is intended for normal system operations, the cashier account is intended for order and payment operations, and the

	viewer account is intended for read-only access.
SQL scripts summary	**Five SQL scripts were prepared to inspect, implement, and verify the database access-control configuration.** **File 1: 00_inspect_database.sql** **The first SQL script was used to inspect the existing PostgreSQL database structure before implementing the access-control configuration.** **The script contains queries that display the available database schemas, existing tables, table columns, sequences, and PostgreSQL roles.** **The inspection focused on identifying:** ● **Existing database schemas.** ● **Existing tables in the public schema.** ● **Table column names and data types.** ● **Existing database sequences.** ● **Existing PostgreSQL users and roles.** **This step was important because the access-control implementation needed to be based on the actual database structure instead of using placeholder table names.** **The inspection also helped identify the database objects that would receive the assigned privileges.** **File 2: 01_create_roles.sql** **The second SQL script was used to create the four PostgreSQL group roles and their corresponding login accounts.** **The following group roles were created:** ● **canteen_admin_role** ● **canteen_app_role** ● **canteen_cashier_role** ● **canteen_readonly_role**

	The following database login accounts were also created: ● **canteen_admin** ● **canteen_app** ● **canteen_cashier** ● **canteen_viewer** **Each login account was assigned to its corresponding group role using PostgreSQL GRANT statements.** **The group roles were created using the NOLOGIN attribute, while the database login accounts were created using the LOGIN attribute.** **This approach allows permissions to be managed through group roles rather than assigning privileges individually to every database account.** **The administrator account is intended for database administration, the application account is intended for normal application operations, the cashier account is intended for transaction-related activities, and the viewer account is intended for read-only access.** **File 3: 02_grant_privileges.sql** **The third SQL script was used to assign the required database permissions to each group role.** **The database used for the implementation was grp08_bwislTdb. Since the database name contains uppercase letters, it was enclosed in double quotation marks in the GRANT CONNECT statement to preserve its exact spelling.** **The four group roles were given CONNECT permission on the Campus Canteen database.** **All four roles were also given USAGE permission on the public schema so they could access database objects for which they have the appropriate privileges.** **The administrator role received ALL PRIVILEGES on all existing tables and sequences in the public schema.** **The application role received SELECT, INSERT, and UPDATE**

	privileges on all existing tables in the public schema. It also received USAGE and SELECT privileges on sequences. **The cashier role received SELECT, INSERT, and UPDATE privileges on all existing tables in the public schema. It also received USAGE and SELECT privileges on sequences to support transaction-related database operations.** **The read-only role received SELECT privileges only on all existing tables in the public schema.** **The use of ON ALL TABLES IN SCHEMA public allows PostgreSQL to assign permissions to all existing tables without manually specifying each table name.** **Similarly, ON ALL SEQUENCES IN SCHEMA public allows privileges to be assigned to all existing sequences in the specified schema.** **These statements make privilege assignment easier when working with a database containing multiple tables and sequences.** **File 4: 03_revoke_privileges.sql** **The fourth SQL script was used to remove unnecessary privileges from restricted roles.** **The read-only role was restricted from performing INSERT, UPDATE, DELETE, and TRUNCATE operations on database tables.** **The application role was restricted from performing DELETE and TRUNCATE operations.** **The cashier role was restricted from performing DELETE, TRUNCATE, REFERENCES, and TRIGGER operations.** **The script also revoked CREATE permission on the public schema from the application, cashier, and read-only roles.** **These restrictions help prevent ordinary application and transaction accounts from performing unnecessary database operations or making unauthorized structural changes.** **The administrator role retains its assigned table and sequence**

	privileges, while the application, cashier, and read-only roles are limited according to their responsibilities. **This helps implement the Principle of Least Privilege by preventing unnecessary access rights.** **File 5: 04_test_privileges.sql** **The fifth SQL script was used to check whether the assigned database permissions were properly applied.** **The script contains queries that display the currently connected database user, identify the current database role, list assigned table privileges, and inspect sequence privileges.** **The verification focused on checking whether:** ● **The administrator has SELECT privileges.** ● **The administrator has INSERT privileges.** ● **The administrator has UPDATE privileges.** ● **The administrator has DELETE privileges.** ● **The application role can view records.** ● **The application role can insert records.** ● **The application role can update records.** ● **The application role cannot delete records.** ● **The cashier role can view records.** ● **The cashier role can insert transaction records.** ● **The cashier role can update records.** ● **The cashier role cannot delete records.** ● **The read-only role can select records.** ● **The read-only role cannot insert records.** ● **The read-only role cannot update records.** ● **The read-only role cannot delete records.** **These checks help confirm that each role follows the intended access-control configuration.** **The verification script displays the assigned privileges. Additional testing using separate login accounts is necessary to confirm that unauthorized operations are actually blocked.**
Verification results	**After preparing the SQL scripts, they were executed in the PostgreSQL**

	Query Tool through pgAdmin 4. **The scripts were organized and executed in the correct order to avoid permission-related errors.** **First Verification: Database Inspection** **The first script was executed to inspect the existing database structure.** **The database name and schema were checked using PostgreSQL queries.** **The following database information was identified:** **Item — Result** **Database Name — grp08_bwislTdb** **Schema Name — public** **Database Management System — PostgreSQL** **Expected result:** **The database inspection queries should successfully display the existing schemas, tables, columns, sequences, and PostgreSQL roles.** **Second Verification: Role Creation** **The second script was executed to create the four group roles and their corresponding login accounts.** **A verification query was used to display the created roles from the PostgreSQL system catalog.** **Expected result:** **All four group roles and their corresponding login accounts should**

	appear in the query results. **The group roles should have the NOLOGIN attribute, while the corresponding user accounts should have the LOGIN attribute.** **Third Verification: Privilege Assignment** **The third script was executed to assign the required database, schema, table, and sequence privileges.** **The execution messages were checked to determine whether the GRANT statements completed successfully.** **Expected result:** **Each role should receive the permissions defined in the access-control plan.** **The administrator should receive full privileges on existing tables and sequences, the application and cashier roles should receive their required data-operation privileges, and the read-only role should receive SELECT privileges only.** **Fourth Verification: Privilege Restrictions** **The fourth script was executed to remove unnecessary privileges from restricted roles.** **Expected result:** ● **The read-only role should not have INSERT privileges.** ● **The read-only role should not have UPDATE privileges.** ● **The read-only role should not have DELETE privileges.** ● **The application role should not have DELETE privileges.** ● **The cashier role should not have DELETE privileges.** ● **The application, cashier, and read-only roles should not have CREATE privileges on the public schema.** ● **The read-only role should retain SELECT privileges.** **Fifth Verification: Permission Checking** **The fifth script was executed to inspect the assigned permissions using PostgreSQL queries.** **The following results represent the expected permission configuration:**

	Verification	Expected Result
	Administrator can SELECT records	TRUE
	Administrator can INSERT records	TRUE
	Administrator can UPDATE records	TRUE
	Administrator can DELETE records	TRUE
	Application can SELECT records	TRUE
	Application can INSERT records	TRUE
	Application can UPDATE records	TRUE
	Application can DELETE records	FALSE
	Cashier can SELECT records	TRUE

Cashier can INSERT records	**TRUE**
Cashier can UPDATE records	**TRUE**
Cashier can DELETE records	**FALSE**
Read-only can SELECT records	**TRUE**
Read-only can INSERT records	**FALSE**
Read-only can UPDATE records	**FALSE**
Read-only can DELETE records	**FALSE**

The expected results demonstrate that the administrator has the intended database management privileges, the application and cashier roles can perform their assigned operations, and the read-only role is restricted to viewing records.

Note: These are expected results based on the prepared SQL scripts, not confirmed execution results. Actual results must be recorded after running the scripts and testing the permissions using the assigned PostgreSQL accounts.

Screenshots

The following screenshots should be included as supporting evidence

	of the implementation and verification process: **Screenshot 1: G8_database_inspection.png** **Screenshot 2: G8_create_roles.png** **Screenshot 3: G8_grant_privileges.png** **Screenshot 4: G8_revoke_privileges.png** **Screenshot 5: G8_test_privileges.png** **The screenshots serve as supporting evidence of the database inspection, role creation, privilege assignment, privilege restrictions, and permission verification.** **Additional screenshots of the Campus Canteen database structure and PostgreSQL database setup may also be included to demonstrate that the existing database was successfully identified and used for the activity.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, several possible issues were considered and addressed.** **Database Name Mismatch** **One of the errors encountered during the preparation of the grant privileges script was:** **ERROR: database "your_database_name" does not exist.** **This happened because your_database_name was used as a placeholder database name and did not match the actual PostgreSQL database name.** **To resolve the issue, the current database name was identified using the following query:** **SELECT current_database();** **The result showed:** **Database Name: grp08_bwislTdb** **Schema Name: public**

	The actual database name was then used in the GRANT CONNECT statement. **Uppercase and Lowercase Database Name** **Another error encountered was:** **ERROR: database "grp08_bwisitdb" does not exist.** **This happened because PostgreSQL converts unquoted identifiers to lowercase. The actual database name contained uppercase letters, specifically IT.** **To resolve this issue, the database name was enclosed in double quotation marks:** **GRANT CONNECT ON DATABASE "grp08_bwisITdb"** **This preserves the exact database name and prevents PostgreSQL from converting its uppercase letters to lowercase.** **Existing Database Roles** **Running role creation commands repeatedly may cause errors if the roles already exist.** **To prevent this issue, the role creation script should be executed only once in a newly prepared database.** **If the roles already exist, they should be checked before executing the CREATE ROLE statements again.** **This helps avoid duplicate role creation errors when executing the SQL scripts.** **Incorrect Execution Order** **Granting privileges before creating the required roles can result in errors.** **To prevent this issue, the scripts were organized and executed in the following order:** **00_inspect_database.sql** **01_create_roles.sql**

	02_grant_privileges.sql **03_revoke_privileges.sql** **04_test_privileges.sql** **This ensures that the existing database structure is inspected first, the required roles are created before permissions are assigned, and the final privileges are checked afterward.** ### Insufficient Database Permissions **Some PostgreSQL commands require appropriate administrative privileges.** **The scripts should be executed using an authorized PostgreSQL administrator account or a database owner with sufficient privileges to ensure that role creation and privilege assignment commands can be completed successfully.** ### Incorrect Table Names **Granting privileges to tables that do not exist can result in relation errors.** **To avoid this issue, the grant privileges script uses:** **ON ALL TABLES IN SCHEMA public** **This allows PostgreSQL to apply the specified privileges to all existing tables in the public schema without manually entering individual table names.** **Similarly, the sequence privilege statements use:** **ON ALL SEQUENCES IN SCHEMA public** **This allows permissions to be assigned to existing sequences in the specified schema.** ### Database Connection and Schema Access **Database-level CONNECT privileges do not automatically provide access to tables.** **The appropriate schema and table privileges must also be granted.**

	To address this, the grant privileges script includes CONNECT on the database, USAGE on the public schema, and the required table and sequence privileges. **Future Database Objects** **Newly created tables and sequences may not automatically receive the same permissions as existing objects.** **Default privileges should be configured for the correct object-owning role when necessary.** **This helps ensure that future database objects receive the intended access permissions.** **Application and Cashier Permission Restrictions** **The application and cashier roles have different responsibilities from the administrator role.** **The application and cashier roles are allowed to view, insert, and update existing database records but are restricted from deleting and truncating records.** **The cashier role should eventually be restricted to the actual order and payment tables required by its responsibilities.** **This helps reduce unnecessary database access and improves the implementation of the Principle of Least Privilege.**
Feedback to owners	**The Campus Canteen Pre-Order and Queue Management System provides database functionality for managing food orders, customer transactions, and queue operations. Implementing Role-Based Access Control can further improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** **1. Maintain separate database roles for administrators, application operations, cashiers, and read-only users.** **2. Avoid using administrator-level database credentials for regular application activities.** **3. Assign actual login accounts to the appropriate group roles.** **4. Use strong passwords and protect database credentials.** **5. Review database permissions whenever new tables or application**

	features are added. 6. Restrict cashier access to the specific order and payment tables required for transaction operations. 7. Use read-only accounts for users who only need to view database information. 8. Apply appropriate database privileges to newly created tables and sequences. 9. Perform regular database backups to help prevent data loss. 10. Avoid granting unnecessary administrative privileges to application accounts. 11. Regularly test database access restrictions to ensure that unauthorized operations are prevented. 12. Keep database credentials and sensitive configuration files out of public GitHub repositories. 13. Review database roles and privileges regularly to prevent unnecessary access. These recommendations can help improve the security, reliability, and maintainability of the Campus Canteen Pre-Order and Queue Management System.
AI usage declaration	Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation. AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, prepare the database access-control plan, and identify possible implementation issues and their corresponding solutions. The generated SQL commands and documentation were reviewed and adjusted to match the available Campus Canteen database information, including the actual database name and schema. The SQL scripts must be validated through PostgreSQL execution using pgAdmin 4. The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results. All passwords and other sensitive credentials must be excluded from the submitted repository, screenshots, and documentation.